Chapter Five

Memory Management

Contents

- Review of physical memory and memory
- Management hardware
- Overlays, swapping, and partitions
- Paging and segmentation
- Placement and replacement policies
- Working sets and thrashing
- Caching

as a glance …

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Register access in one CPU clock (or less)
- Main memory can take many cycles
- Cache sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

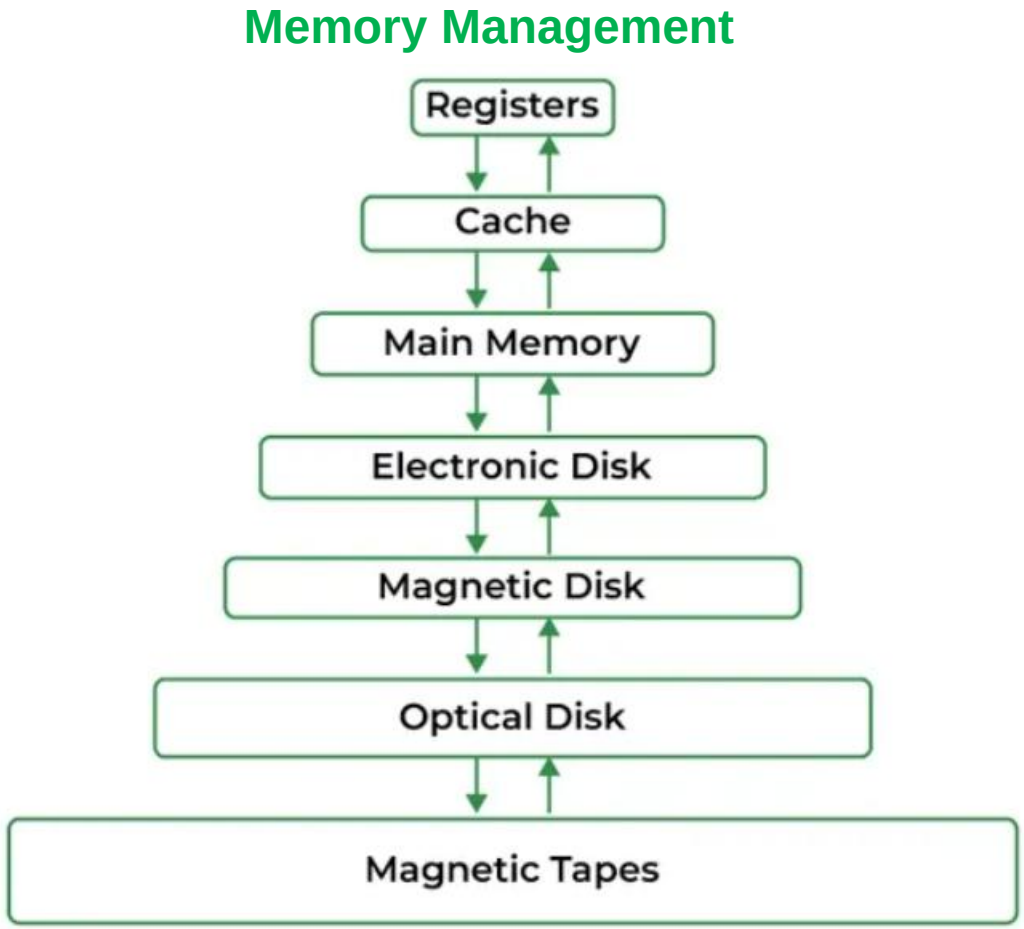
Review of Physical Memory & Memory

- Memory management is the process of controlling and organizing a computer's memory by allocating portions, called blocks, to different executing programmes to improve the overall system performance.
  - ✓The most important function of an operating system is to manage primary memory.
  - ✓Supports multiple processes simultaneously in memory.
  - ✓Protects processes from unauthorized access.
  - ✓Enables swapping and virtual memory efficiently.

Physical memory (RAM): Volatile and fast

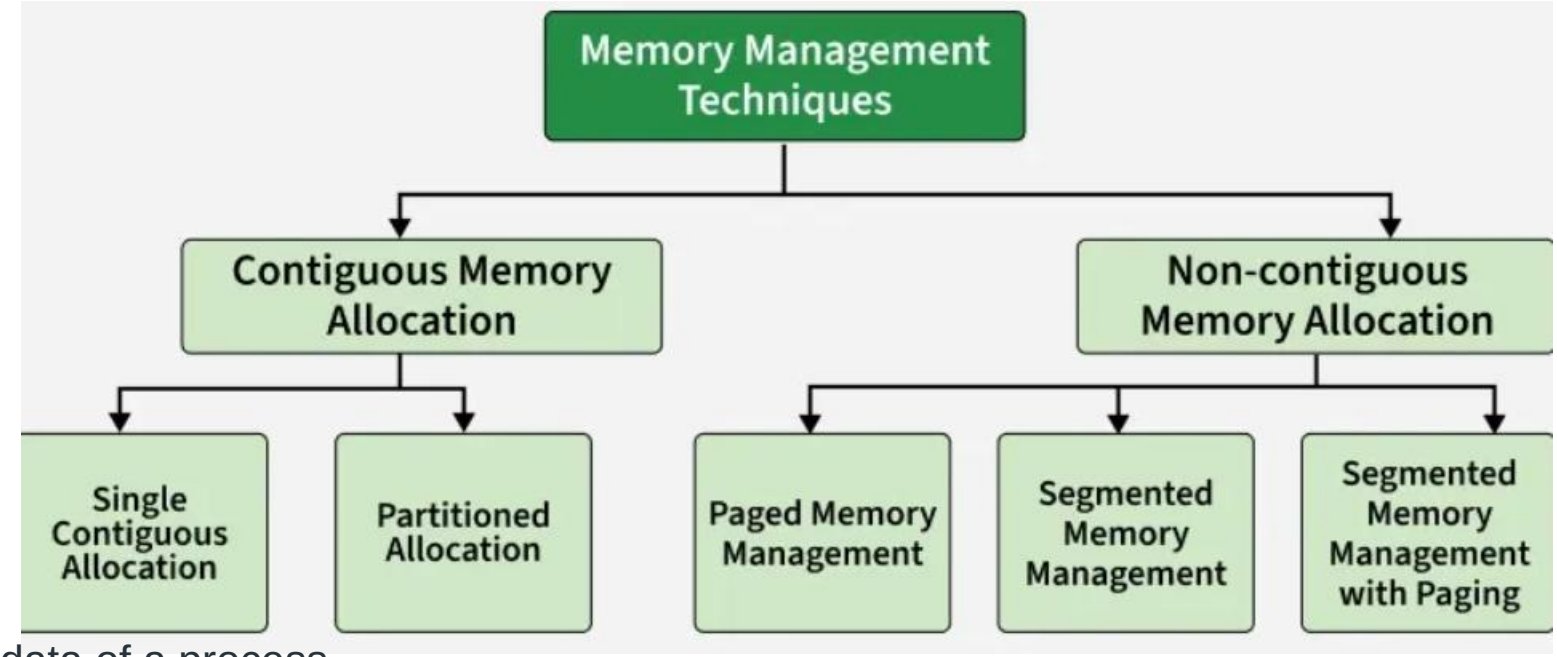
Memory hierarchy =

Registers → Cache → Main Memory → Secondary Storage



...cont

- **Techniques in Memory Allocation:** Used by an operating system to efficiently allocate, utilize, and manage memory resources for processes. Various techniques help the operating system manage memory effectively.



Contiguous Memory Allocation: All instructions and data of a process are stored in adjacent memory locations.

Simplest form of memory management. In this technique, the main memory is divided into two parts:

Single Contiguous Memory Allocation: One part is reserved for the Operating System

The remaining part is allocated to a single user process

Partitioned Memory Allocation: Main memory is divided into multiple contiguous partitions, and each partition can hold one process. This technique supports multiprogramming.

Non-Contiguous Memory Allocation: Memory management technique in which a process is divided into smaller parts and these parts are stored in different, non-adjacent locations in main memory. Unlike contiguous allocation, the entire process does not need to be placed in a single continuous block of memory. This technique is widely used in modern operating systems because it improves memory utilization and reduces fragmentation problems.

Memory Management Hardware

- In computer architecture, memory management refers to the process of managing and organizing the memory resources of a computer system.
- Memory management hardware includes various components like memory controllers, memory caches, and translation lookaside buffers (TLBs). These components work together to optimize memory access, improve performance, and ensure the smooth running of programs.
- The memory controller is responsible for managing the flow of data between the central processing unit (CPU) and the main memory. It coordinates the transfer of data, handles read and write operations, and ensures data integrity.
- Memory caches act as a buffer between the CPU and the main memory, storing frequently used data and instructions for quick access.
- TLBs are small high-speed memory caches that store recently used memory addresses and their corresponding physical locations. They help speed up the translation of virtual memory addresses to physical memory addresses.

…Base Limit Registers

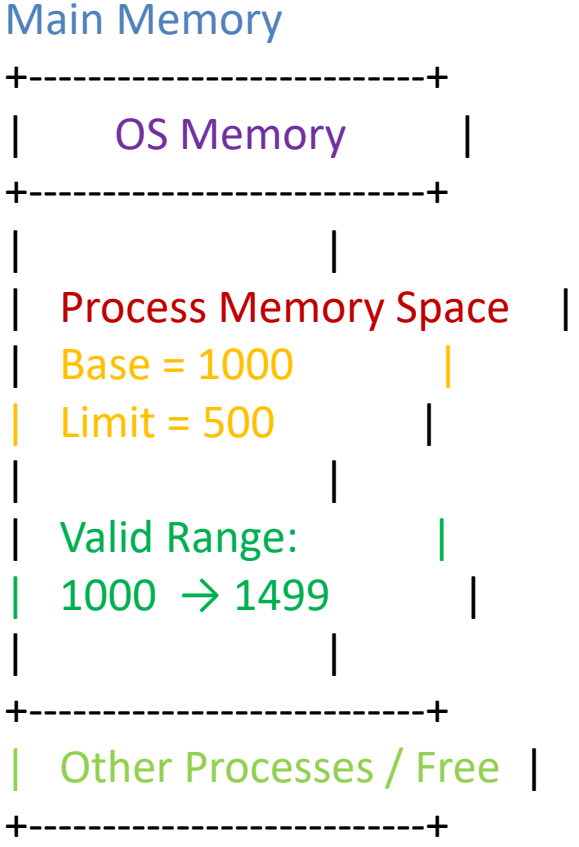
- **Base and Limit registers** are simple hardware-supported mechanisms used by an operating system to provide **memory protection** and **process isolation**.

•**What is the Limit Register?**
•The **Limit Register** stores the **size (range)** of memory allocated to the process.
•It defines **how much memory** the process can access.

•Base register → **starting address**
•Limit register → **size of memory**
•Used for **memory protection**
•Common in **early operating systems**

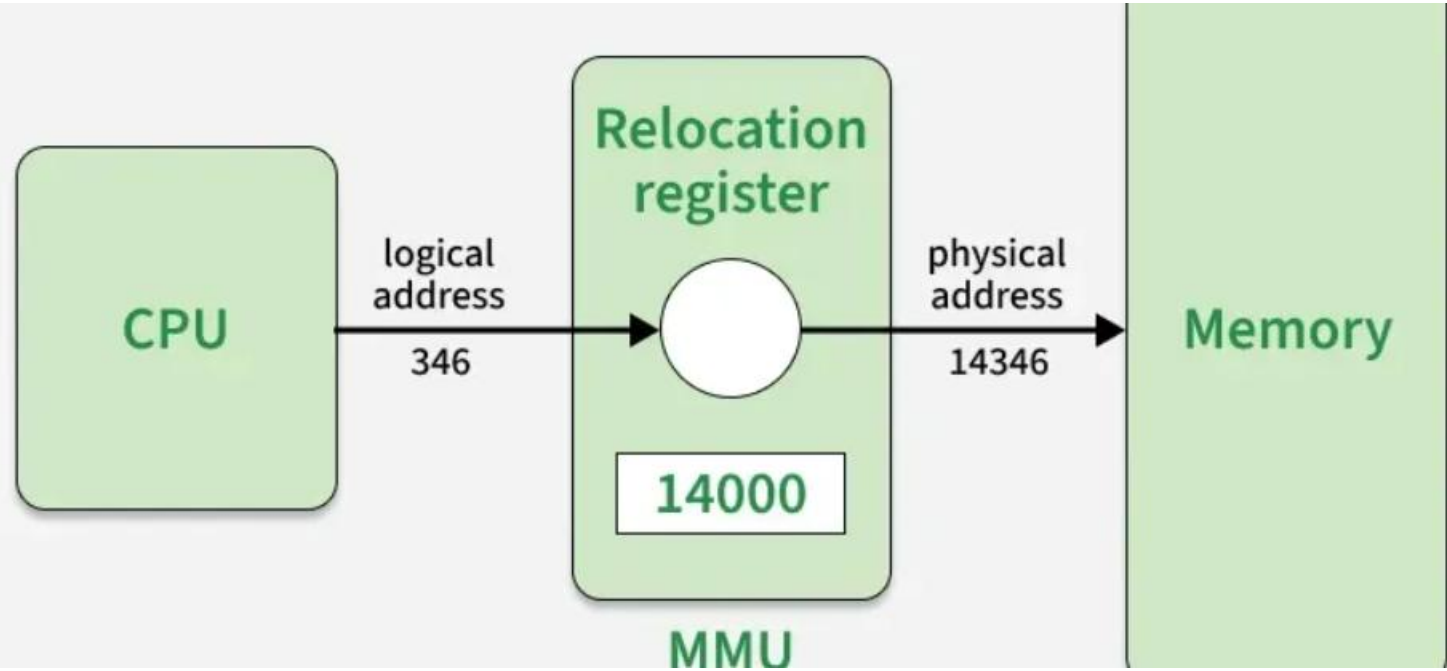
- **How Base and Limit Registers Work Together?**
- For every memory access made by a process:
  - ✓ The CPU generates a **logical address**.
  - ✓ The OS/hardware checks: 0 ≤ Logical Address < Limit
  - ✓ If valid, the **physical address** is calculated as: Physical Address = Base + Logical Address
  - ✓ If invalid, a **memory protection fault (trap)** occurs.

Memory Layout Illustration



… logical to physical address

- **Memory Management Unit:** The Memory Management Unit (MMU) is a hardware component in the computer system that handles all memory and caching operations associated with the CPU. Its main function is to translate logical (virtual) addresses generated by the CPU into physical addresses in main memory.



- **Important Points:**
  - ✓ Logical addresses provide abstraction, so processes don't need to know physical locations.
  - ✓ These logical addresses are translated into physical addresses using a page table.
  - ✓ Translation is transparent and handled by hardware (MMU).
  - ✓ Enables efficient memory management through paging and segmentation.

… Logical Address vs. Physical Address

Logical Address	Physical Address
Generated by the CPU during program execution	Generated by the Memory Management Unit (MMU)
Logical Address Space is set of all logical addresses generated by CPU in reference to a program.	Physical Address is set of all physical addresses mapped to the corresponding logical addresses.
User can view and access the logical address of a program.	User can never view physical address of program.
Can change during program execution (due to relocation, paging, etc.)	Generally fixed once assigned in memory
The user can use the logical address to access the physical address.	The user can indirectly access physical address but not directly.
Logical address can be change.	Physical address will not change.
Virtual address.	Real address.

Overlays, Swapping & Partitions

Overlays:

- **Overlays** are a memory management technique used when a program is **larger than the available main memory**.
  Only the **required part of the program** is loaded into memory at a time, while the rest remains on disk.
- Mainly used in **early systems** with limited RAM.

How it Works:

- ✓ Program is divided into **modules (overlays)**.
- ✓ Only one overlay is loaded into a **fixed memory region** at a time.
- ✓ When another function is needed, the current overlay is replaced.

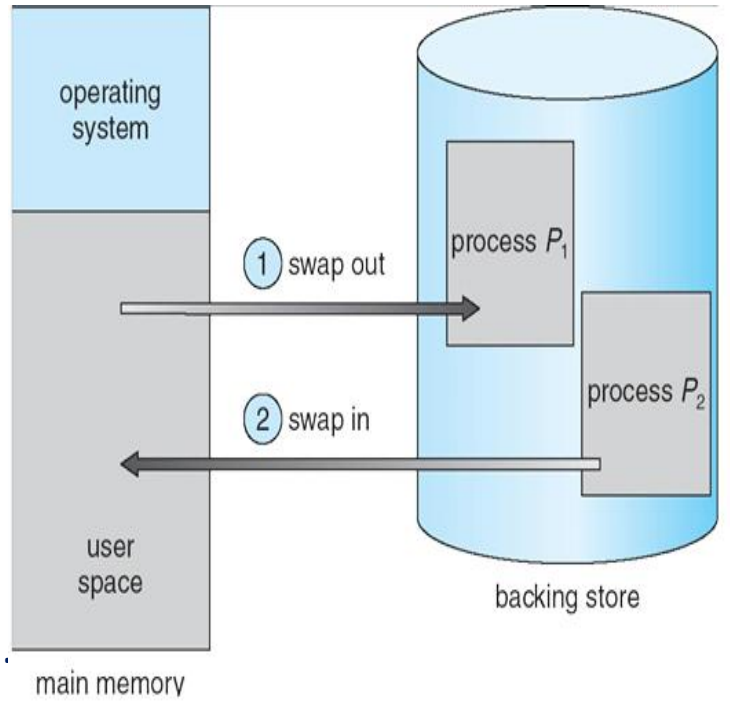
Need for Overlays:

- In early computing systems, overlays are commonly used due less amount of storage capacity for RAM. Overlays are primarily used to for-
  - ✓ Running programs larger than the available RAM storage.
  - ✓ Ensuring maximum utilization of memory resources.
  - ✓ Allowing multiple programs to run simultaneously by sharing memory.

...cont

Swapping:

- It is a mechanism in which a process can be moved temporarily from main memory to secondary storage (disk) and make that memory available to other processes.
- At some later time, the system swaps back the process from the secondary storage to main memory
- This allows the operating system to manage limited RAM effectively and run multiple processes concurrently in a multiprogramming environment.
  - ✓ Reduces memory fragmentation by moving processes in and out of RAM.
  - ✓ Allows large processes to run even if they exceed available RAM.
  - ✓ Frees RAM temporarily by suspending inactive or waiting processes.
  - ✓ Supports virtual memory to give processes more memory than physically available.



How it Works:

- A process is moved into RAM from disk when it is ready to execute.
- After the process has run for a while (or if higher-priority processes need memory), it may be temporarily swapped out to disk.
- The CPU scheduler decides which process to swap in and out, often based on priority or scheduling policies.
- When the swapped-out process is needed again, it is swapped back into memory to continue execution.

… cont

- **Partitioning** divides main memory into **fixed or variable-sized regions**, where each region holds **one process**.
- **Fixed Partitioning:**
  - ✓ Memory is divided into **fixed-size partitions**
  - ✓ Simple but causes **internal fragmentation**
- **Variable Partitioning:**
  - ✓Partitions are created **dynamically**
  - ✓Reduces internal fragmentation
  - ✓Causes **external fragmentation**

Feature	Partitions	Overlays	Swapping
Who manages it?	Operating System	The Programmer	Operating System
Main Goal	Multiprogramming	Running massive code in tiny RAM	Freeing up RAM for active tasks
Efficiency	Low (Fragmentation)	High (but complex to code)	Medium (Disk I/O is slow)
Modern Use?	Limited (replaced by Paging)	Very Rare (mostly Embedded)	Very High (Virtual Memory)

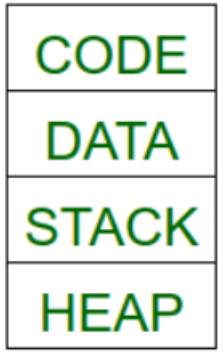
paging and segmentation

- Paging is a method or technique which is used for non-contiguous memory allocation.
- It is a fixed-size partitioning theme (scheme), both main memory and secondary memory are divided into equal fixed-size partitions.
- The partitions of the secondary memory area unit and main memory area unit are known as pages and frames respectively.

- **Segmentation** is another non-contiguous memory allocation scheme, similar to paging. However, unlike paging which divides a process into fixed-size pages segmentation divides memory into variable-sized segments that correspond to logical units such as functions, arrays, or data structures.

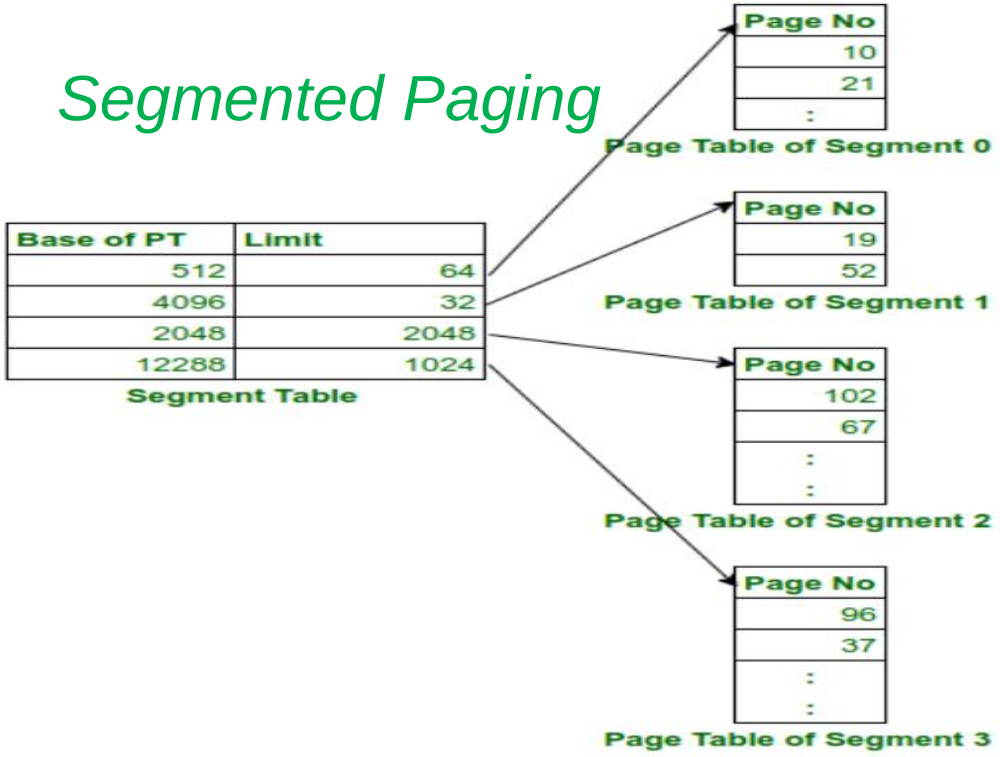
•A process is generally divided into four segments, namely - code, data, stack and heap.

Segments of a process



•The memory management unit (MMU) first checks the segment table → finds the base of the page table → then uses the page number to locate the frame → finally combines with offset to form the physical address.

Segmented Paging



...cont

Sr. No.	Key	Paging	Segmentation
1	Memory Size	In Paging, a process address space is broken into fixed sized blocks called pages.	In Segmentation, a process address space is broken in varying sized blocks called sections.
2	Accountability	Operating System divides the memory into pages.	Compiler is responsible to calculate the segment size, the virtual address and actual address.
3	Size	Page size is determined by available memory.	Section size is determined by the user.
4	Speed	Paging technique is faster in terms of memory access.	Segmentation is slower than paging.
5	Fragmentation	Paging can cause internal fragmentation as some pages may go underutilized.	Segmentation can cause external fragmentation as some memory block may not be used at all.
6	Logical Address	During paging, a logical address is divided into page number and page offset.	During segmentation, a logical address is divided into section number and section offset.
7	Table	During paging, a logical address is divided into page number and page offset.	During segmentation, a logical address is divided into section number and section offset.
8	Data Storage	Page table stores the page data.	Segmentation table stores the segmentation data.

Placement and replacement policies

- Placement policies determine where an incoming process or page should be put in the available main memory. This is crucial for minimizing **External Fragmentation** (those tiny "holes" of wasted space).
- The three most common algorithms are:
  - ✓ **First Fit:** The OS scans memory from the beginning and puts the process in the **first** hole that is big enough. It's fast but can clutter the beginning of the memory.
  - ✓ **Best Fit:** The OS scans the *entire* list of holes and picks the **smallest** one that fits the process. This leaves the most large blocks free, but it creates many tiny, useless fragments.
  - ✓ **Worst Fit:** The OS puts the process in the **largest** available hole. The logic is that the leftover space will still be large enough to hold another process.

… Replacement policies

- When memory (or the cache) is full and the OS needs to bring in a new page from the disk, it must choose an existing page to evict. This is the **Replacement Policy**.
- The goal here is to minimize **Page Faults** (when the CPU looks for a page that isn't in RAM).

Policy	Logic	Pros/Cons
FIFO	The oldest page in memory is the first to go.	Simple to implement, but can kick out "heavy-hitter" pages that are still being used.
LRU (Least Recently Used)	Replaces the page that hasn't been touched for the longest time.	Very efficient; based on the idea that if you haven't used it lately, you probably won't soon.
Optimal (OPT)	Replaces the page that won't be used for the longest time in the *future*.	**Theoretical only.** Since the OS can't see the future, this is just a benchmark for other algorithms.
LFU (Least Frequently Used)	Replaces the page with the lowest "hit" count.	Good for static data, but fails if a page was used a lot early on and is never used again.

Working sets and thrashing

- In operating systems, **Thrashing** is the **"worst-case scenario"** for performance, and the **Working Set** is the primary strategy we use to prevent it.

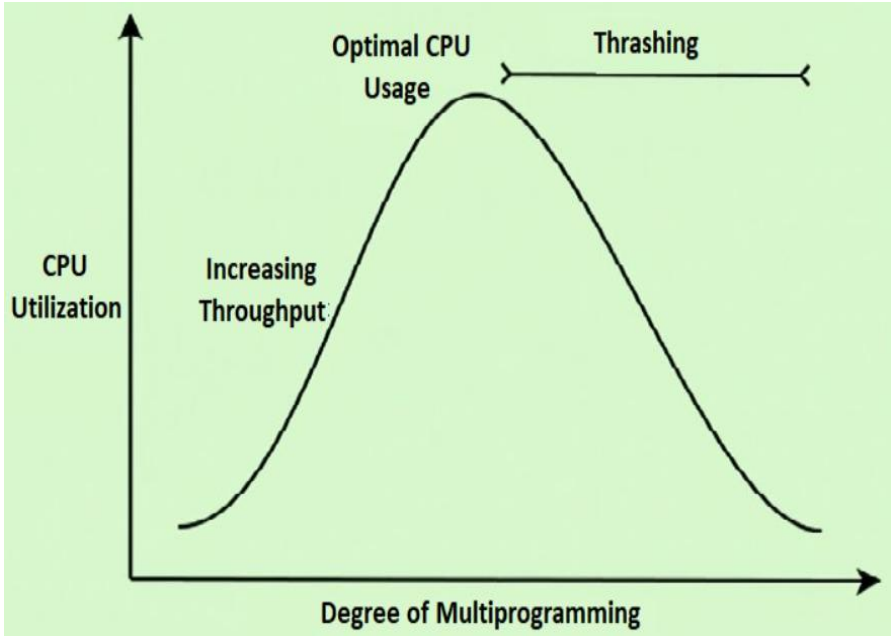
What is Thrashing?

- **Thrashing** occurs when the operating system spends more time swapping pages in and out of memory than actually executing instructions.
- It's like trying to cook a 10-ingredient meal in a kitchen that only has room for 2 ingredients at a time. You spend all your energy running to the pantry (Disk) and back to the counter (RAM), and the actual cooking (CPU processing) never happens.
- What Causes Thrashing?

  ✓ **High Degree of Multiprogramming**

  ✓ **Improper Page Replacement Algorithms**

  ✓ **Less Physical Memory**



...cont

- To stop thrashing, the OS uses the **Working Set** concept. This is based on the **Locality of Reference**: at any given point in time, a process doesn't need its entire code—it only needs a small "cluster" of pages to function efficiently.
  - ✓ **Definition:** The Working Set is the set of pages a process has used in the most recent Δ (delta) time units.
  - ✓ **The Goal:** If the OS can keep the entire "Working Set" of a process in RAM, the page fault rate will stay low.

How the OS Uses It:

1. The OS calculates the **Working Set Size (WSS_i)** for every active process.
2. It calculates the **Total Demand (D):**

$$D = \sum WSS_i$$

3. **The Rule:** * If $D > \text{Total Available RAM}$, the OS knows Thrashing is imminent.
   - **The Solution:** Instead of letting everyone suffer, the OS will "suspend" (swap out) one entire process to free up enough room for the remaining processes to have their full working sets.

Comparison: Thrashing vs. Healthy Paging

Feature	Healthy Paging	Thrashing
CPU Utilization	High	Extremely Low
Disk Activity	Occasional	Constant/Maxed out
System Feel	Snappy	"Frozen" or Lagging
Cause	Normal multitasking	Over-allocation of memory

Caching

- In an Operating System, **Caching** is the strategy of storing frequently accessed data in a smaller, faster storage layer so that future requests for that data can be served much quicker.
- It is based on the **Principle of Locality**: if you use a piece of data once, you're likely to use it (or the data right next to it) again very soon. Use "Least recently used" policy

Key Points:

- ✓Stores frequently used data in a fast storage area.
- ✓Reduces time wasted in accessing data from a slower hard disk.
- ✓Provides quick access to recently used information.
- ✓Makes websites and applications load faster on repeated use.
- ✓Helps improve overall system performance.

...cont

Types of Caching in OS:

- **Caching isn't just a hardware thing; the OS manages several "software" caches to speed up performance:**
  - ✓ **Disk Buffer Cache:** The OS keeps a portion of RAM dedicated to storing recently read sectors from the hard drive. If you open a large file, close it, and immediately open it again, it feels instant because the OS is serving it from the **Disk Cache** in RAM.
  - ✓ **TLB (Translation Lookaside Buffer):** This is a specialized hardware cache used by the Memory Management Unit (MMU). It stores recent "Virtual-to-Physical" address translations. Without the TLB, every single memory access would require two trips to RAM—one to look up the address map and one to get the data.
  - ✓ **Web/Application Caching:** Browsers and OS services cache DNS records and web elements (like images) locally so they don't have to be re-downloaded over the network.

Advantages of Caching:

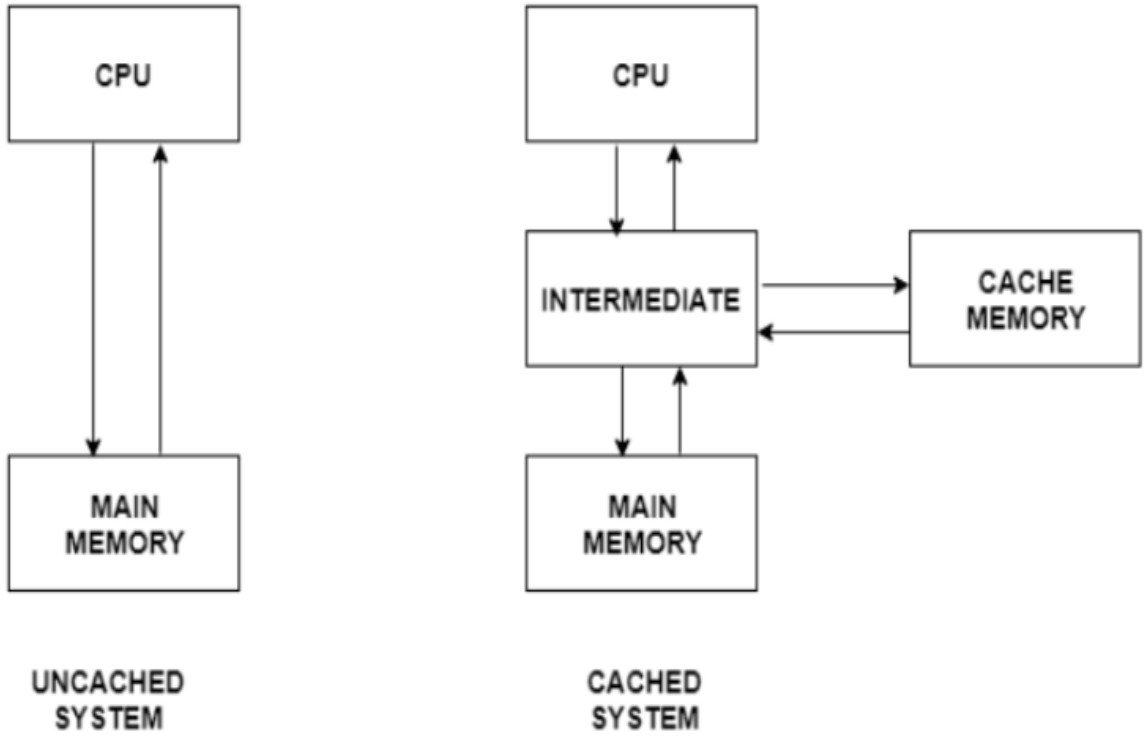
- ✓ It is faster for accessing the data as compared to RAM.
- ✓ It enhances the performance of the application.
- ✓ It reduces the Server load.

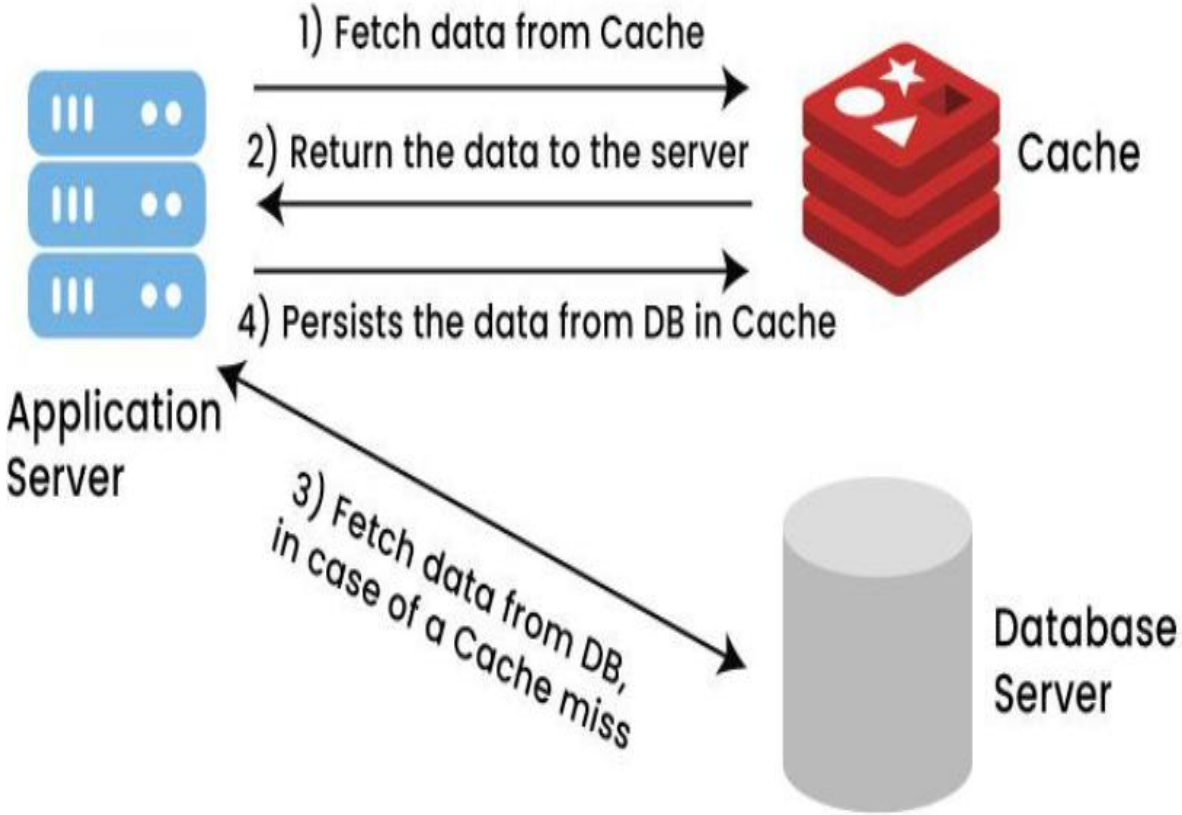
Disadvantages of Caching:

- ✓ Data is lost when the system is turned off.
- ✓ The cost is high as compared to other memory.

… as summary

- Caching is essentially a **"shortcut"** for the CPU. Without it, even the fastest processor in the world would spend most of its time sitting idle, waiting for the relatively slow RAM to send over data.





Thank You!

?